```
begin
    using Oscar: ZZ, matrix_space, polynomial_ring, degree
end
```

[0 4 2 999999 999999 999999; 999999 0 999999 5 999999 999999; 999999 999999 0 8 10 999999; 999999 999999 999999 0 999999 2; 999999 999999 999999 999999 0 3; 999999 999999 999999 999999 999999 0]

```
begin
    # Global Data-Oriented Defaults (Immutable Constants)
    Num_Nodes = ZZ(6)
    Start_Node = ZZ(1)
    End_Node = ZZ(6)
    Alg_Inf = ZZ(999999)

    Space_Z = matrix_space(ZZ, Int(Num_Nodes), Int(Num_Nodes))

    W_Alg = Space_Z(map(ZZ, [
        0       4       2       Alg_Inf Alg_Inf Alg_Inf;
        Alg_Inf 0       Alg_Inf 5       Alg_Inf Alg_Inf;
        Alg_Inf Alg_Inf 0       8       10      Alg_Inf;
        Alg_Inf Alg_Inf Alg_Inf 0       Alg_Inf 2;
        Alg_Inf Alg_Inf Alg_Inf Alg_Inf 0       3;
        Alg_Inf Alg_Inf Alg_Inf Alg_Inf Alg_Inf 0
    ]))
end
```

Lecture 23: Shortest Paths and Algorithm Complexity

I. Efficient Shortest Paths with Dijkstra's Algorithm

Optimal Path Cost: 11

- **Mathematical Proof Aggregation**: true

```
begin
    let
        # 1. Pure FP Focus: Stateless Foldl Dijkstra
        dijkstra_step = (state, _) -> begin
            unv, dist = state
            isempty(unv) && return state
            u = reduce((min_n, n) -> dist[n] < dist[min_n] ? n : min_n, unv)
            dist[u] == Alg_Inf && return state
            new_unv = filter(!=(u), unv)
            new_dist = copy(dist)
            foldl(new_unv, init=new_dist) do d_acc, v
                w = W_Alg[u, v]
                if w != Alg_Inf && dist[u] + w < d_acc[v]
                    d_acc[v] = dist[u] + w
                end
                d_acc
            end
            (new_unv, new_dist)
        end

        init_state = (Set(1:Int(Num_Nodes)), Dict(i => (i == Int(Start_Node) ? ZZ(0)
        : Alg_Inf) for i in 1:Int(Num_Nodes)))
        _, fp_dist = foldl(dijkstra_step, 1:Int(Num_Nodes), init=init_state)
        sec1_fp = fp_dist[Int(End_Node)]

        # 2. Abstract Algebra Focus: Tropical Semiring Exponentiation
        tropical_mul = (A, B) -> begin
            Space_Z([minimum([A[i, k] + B[k, j] for k in 1:Int(Num_Nodes)]) for i in
            1:Int(Num_Nodes), j in 1:Int(Num_Nodes)])
        end
        aa_all_pairs = foldl((acc, _) -> tropical_mul(acc, W_Alg), 1:
        (Int(Num_Nodes)-1), init=W_Alg)
        sec1_aa = aa_all_pairs[Int(Start_Node), Int(End_Node)]

        # 3. Geometric/Group Theory Focus: Topological Bellman-Ford Vector Projection
        relax_edges = (V, _) -> [minimum([V[k] + W_Alg[k, j] for k in
        1:Int(Num_Nodes)]) for j in 1:Int(Num_Nodes)]
        init_V = [i == Int(Start_Node) ? ZZ(0) : Alg_Inf for i in 1:Int(Num_Nodes)]
        geo_dist = foldl(relax_edges, 1:(Int(Num_Nodes)-1), init=init_V)
```

```
        sec1_grp = geo_dist[Int(End_Node)]

        Markdown.parse("""
        **Optimal Path Cost:** $(sec1_fp)

        * **Mathematical Proof Aggregation**: $(sec1_fp == sec1_aa == sec1_grp)
        """)
    end
end
```

II, III, IV. Complexity Classes: P vs NP and Solvability Bounds

Complexity Bounding: P $\mathcal{O}(n^k)$ scales vastly slower than NP $\mathcal{O}(n!)$ for $n = 5$.

- **Mathematical Proof Aggregation**: false

```
begin
    let
        n_val = ZZ(5)

        # 1. Pure FP Focus: Mapping polynomial vs exponential bounds
        fp_p_bound = n_val^3
        fp_np_bound = foldl(*, 1:Int(n_val), init=ZZ(1)) # Factorial O(n!)
        sec2_fp = fp_p_bound < fp_np_bound

        # 2. Abstract Algebra Focus: Ring Dimensions
        R, x = polynomial_ring(ZZ, "x")
        aa_p_space = degree(x^3) # Polynomial degree
        aa_np_space = 2^Int(n_val) # Boolean satisfiability combinatorial space
        sec2_aa = aa_p_space < aa_np_space

        # 3. Geometric/Group Theory Focus: Spanning Trees vs Hamiltonian Cycles Space
        geo_p_edges = n_val - 1 # Minimum Spanning tree edges
        geo_np_edges = (n_val * (n_val - 1)) / 2 # Complete graph edges scaling
        combinations
        sec2_grp = geo_p_edges < geo_np_edges

        Markdown.parse("""
        **Complexity Bounding:** P \$\\mathcal{O}(n^k)\$ scales vastly slower than NP
        \$\\mathcal{O}(n!)\$ for \$n=$(n_val)\$.

        * **Mathematical Proof Aggregation**: $(sec2_fp == sec2_aa == sec2_grp)
        """)
    end
end
```

VII & VIII. Implications & Routing Applications

Maximum Base Edge Cost in Routing Protocol: 10

- **Mathematical Proof Aggregation**: true

```
begin
    let
        # Evaluating a Network Topology Routing Metric (Diameter/Reachability)

        # 1. Pure FP Focus: Maximum routing path length reduction
        w_matrix = [Int(W_Alg[i,j]) for i in 1:Int(Num_Nodes), j in 1:Int(Num_Nodes)]
        sec3_fp = foldl((max_val, val) -> (val != 999999 && val > max_val) ? val :
        max_val, w_matrix, init=0)

        # 2. Abstract Algebra Focus: Ring Extrema evaluation
        sec3_aa = Int(maximum([W_Alg[i,j] for i in 1:Int(Num_Nodes), j in
        1:Int(Num_Nodes) if W_Alg[i,j] != Alg_Inf]))

        # 3. Geometric/Group Theory Focus: Adjacency Norm (Max Edge projection)
        sec3_grp = maximum(filter(!=(Int(Alg_Inf)), reshape(w_matrix, :)))

        Markdown.parse("""
        **Maximum Base Edge Cost in Routing Protocol:** $(sec3_fp)

        * **Mathematical Proof Aggregation**: $(sec3_fp == sec3_aa == sec3_grp)
        """)
    end
end
```